Bounded-memory streaming redaction of personal data and credentials with a whole-document equivalence guarantee

Mounish Senisetty[1], Sandhiya Kennedy[1], Ashrith Reddy N[1], Parvathy R[1*]

[1*]Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India.

*Corresponding author(s). E-mail(s): parvathyr@am.amrita.edu;
Contributing authors: am.ai.u4aid23058@am.students.amrita.edu;
am.ai.u4aid23056@am.students.amrita.edu;
am.ai.u4aid23014@am.students.amrita.edu;

Abstract

Logs and data exports carry personal data and credentials and often exceed memory. Chunked processing adds a silent failure: a value straddling a chunk boundary reaches the detector in two halves and is written out in the clear. SafeStream-Redactor, a local redaction engine, treats chunking as a correctness problem. Each detector declares its longest span and the context it reads. We show that an overlap as long as the longest span can still leak, and prove that if the overlap covers every detector's span plus its right context, the streamed output equals a whole-document pass, parallel output equals sequential output or falls back to it, and the working buffer is independent of input size; the engine enforces this overlap. Naive 1 KiB chunking leaked 237 RedactionBench values that a whole-document pass redacts and missed 388 in real system logs; overlapping windows still missed some, SafeStream none. Streaming added 8.4–8.54 MiB from 8 MiB to 2 GiB, against 904.6 MiB for whole-document redaction of 32 MiB; throughput reached 10.5 MB/s with one worker and 41.1 MB/s with four. On held-out data the pattern tiers fully redacted 0.437 of credentials, against 0.311 for gitleaks and 0.270 for TruffleHog; the gitleaks margin survives clustered resampling and disabled filters. Detection remains incomplete: on 2,000 real SSH log lines the pattern tiers redacted every IP address but no user name, and no configuration fully protected more than 17.9% of RedactionBench documents.

Keywords: Data redaction, Secret detection, Data loss prevention, Streaming algorithms, Bounded memory, Personally identifiable information

1 Introduction

Text artefacts that leave an organisation, such as an application log attached to a support ticket, a database export handed to an analyst, or a packet capture shared with a vendor, routinely carry two classes of sensitive value: personal data (e-mail addresses, telephone and card numbers, national identifiers, names) and credentials (application programming interface keys, access tokens, private keys, passwords). Data-loss-prevention systems inspect such content before it crosses a boundary [1], and measurement studies show that credentials leak into shared artefacts at scale [2, 3].

Two families of tools address the problem. Personal-data engines such as Microsoft Presidio [4] combine pattern recognisers with a statistical named-entity model but ship no credential recognisers. Secret scanners such as gitleaks [5] and TruffleHog [6] target credentials in source repositories, and their accuracy varies widely across secret types [7]. The artefacts that most need redaction are often too large to analyse in one call, so they are processed in pieces. A chunked redactor can then fail silently: a value that straddles a chunk boundary is presented to the detector in two halves, neither of which matches, and both halves are written out in the clear. The same problem is long established in network security, where signatures split across packets motivated stream reassembly in intrusion detection [8] and streaming modes in regular-expression engines [9, 10]. In redaction, however, detectors read context around a match ("Patient ID:" before a number, a key name before a token), confidences are adjusted by neighbouring text, and overlapping detections are merged before the text is rewritten. Overlapping windows are the obvious remedy, but how much overlap is enough depends on all of these, and the obvious answer, an overlap as long as the longest match, is wrong: a match whose right context is cut off at the window edge can be rejected there, so the boundary does not retreat out of it and its first part is emitted in the clear. None of the redaction or secret-scanning systems we examined states when its chunked output equals its whole-document output.

This paper answers that question for a concrete system, SafeStream-Redactor (hereafter SafeStream). Its contributions are the following.

1. **A tight, per-detector overlap condition for context-dependent redaction.** Every detector d declares an upper bound L_d on the span it can emit and bounds $C_{\mathrm{left},d}$ and $C_{\mathrm{right},d}$ on the context it reads (Assumptions A1 and A2). We prove that an overlap $w \geq \max_d \max(L_d + C_{\mathrm{right},d},\, C_{\mathrm{left},d})$ makes the streamed output equal to the whole-document output for every chunking, and show that the term $L_d + C_{\mathrm{right},d}$ cannot be dropped: an overlap of $\max_d L_d$, the natural choice, can leak (Section 5). We also measure the failure the condition prevents: on the test corpora, naive chunking leaked up to 237 gold values that a whole-document pass redacts, and overlapping windows still leaked some, while SafeStream's output was identical to the whole-document output under every chunking tried (Section 7.1).

The underlying locality argument is standard; what is new is the condition for pipelines whose detectors read context on both sides, adjust confidences and resolve overlaps jointly, which pattern-matching engines that carry automaton state across blocks do not address.

2. **A guarantee enforced by the code, not chosen by hand.** The engine computes the condition from the detectors' declarations and refuses to run with a smaller overlap; regression and differential tests check that the declarations hold. For the default pipeline the requirement is exactly the default overlap of 8192 characters.
3. **Parallel redaction that cannot lose or duplicate text.** A seam protocol between worker processes makes the parallel output equal to the sequential output under the same condition, and falls back to sequential processing on any disagreement, so no character is lost or emitted twice whatever the detectors do (Appendix A).
4. **Bounded memory, measured.** The working buffer is bounded independently of input size, and peak memory stays flat across a 256-fold range of input sizes, where whole-document redaction needs about 28 times the input size (Section 7.2).
5. **A detection evaluation that separates robust from fragile margins.** Equivalence says nothing about how good the detectors are. We evaluate them on held-out test splits against Presidio and two secret scanners, with paired bootstrap intervals at the document, cluster and two-stage levels, a scanner sensitivity run with false-positive filters disabled, an ablation and a contamination analysis (Sections 6 and 7). Two margins survive every check: a higher fully redacted rate for credentials than gitleaks, and higher character F1 than Presidio with the same spaCy model. Others do not, and we report them as such.

Two scope limits are stated at the outset. First, the guarantee concerns equivalence rather than accuracy: streaming does not improve detection, it makes the streamed result the same as the whole-document result, so every detection error of the whole-document pipeline is also an error of the streamed one. Second, the evaluation has known weaknesses that we quantify rather than omit: one corpus, RedactionBench, was inspected during rule development (Section 7.5); two corpora are synthetic; and fewer than one RedactionBench document in five was fully protected by any configuration we evaluated (Section 7.5). Section 8 states when the approach is useful despite these limits and what evidence a deployment would need.

2 Background and related work

Personal-data detection and de-identification

Rule-based de-identification of clinical text dates back to lookup tables and regular expressions applied to nursing notes [11]; Philter refined the approach on a large annotated clinical corpus [12]. The 2014 i2b2/UTHealth shared task established annotated corpora and evaluation practice [13], and recurrent neural networks reached strong results on them [14]. Presidio [4] combines pattern recognisers with spaCy or transformer models and ships an evaluation toolkit with a synthetic dataset [15]. Lison et al. argue that sequence labelling alone does not amount to anonymisation and

call for explicit disclosure-risk measures [16]; the Text Anonymization Benchmark operationalises this for court judgments [17]. Named-entity recognition underpins the free-text part of these systems, from the CoNLL-2003 shared task [18] to transformer encoders [19, 20] and generalist span models [21]. Pseudonymisation with a keyed hash such as HMAC [22] preserves referential integrity but reveals equality between values, so it is not anonymisation in the sense of k-anonymity [23]. Redaction as studied here is narrower than anonymisation: it removes spans that detectors recognise and makes no claim about re-identification from what remains.

Credential and secret detection

Meli et al. measured secret leakage across public GitHub repositories [2]. Approaches range from vendor-specific regular expressions [24] to machine-learned filters that reduce false positives [25] and neural detectors for free-form passwords [26]. SecretBench provides a labelled corpus of candidate secrets [27], and a comparison of nine tools attributed false positives to generic patterns and entropy heuristics and false negatives to missing rules [7]. Krause et al. studied how developers handle leaked secrets [3]. Shannon entropy [28] is the usual signal for undocumented high-entropy secrets. The Nosey Parker rule set [29] supplies the labelled examples we evaluate on.

Detection across segment boundaries

That a pattern split across units of input can evade a detector is well known in network security: Ptacek and Newsham showed that intrusion detectors which do not reconstruct streams as the end host does can be evaded [8], and Hyperscan [9] provides a streaming mode whose matches do not depend on how the data is split into blocks [10]. These systems carry automaton state across boundaries, which makes pattern matching exact but says nothing about what happens after a match: validators, context-dependent confidences and joint overlap resolution all read text the automaton has already consumed. A redaction pipeline differs in three ways that affect correctness: detectors consult bounded context on both sides of a match, confidences depend on nearby text, and overlapping detections are resolved jointly before the output is rewritten. Overlapping windows are a common way to process long inputs, and the stream model [30] formalises computation in bounded memory; linear-time regular-expression matching goes back to Thompson [31]. Among the redaction, de-identification and secret-scanning systems cited above, we found none that states when chunked output equals whole-document output or how the overlap must be sized. We do not claim that the underlying locality argument is new; the contribution is its instantiation and enforcement for context-dependent redaction.

3 Problem statement and threat model

Definition 1 (Detections and redaction) A text T is a finite character string, and $T[a{:}b]$ is its half-open substring. A *detection* is a tuple (i, j, τ, c) with $0 \le i < j \le |T|$: a half-open span, an entity type and a confidence in $[0, 1]$. Each detection is produced by one detector d of a finite set $\mathcal{D}$. A *pipeline* Π maps T to a set of pairwise-disjoint detections. *Redaction* rewrites T by replacing each span in $\Pi(T)$ according to a policy (replace, mask, hash, keyed

pseudonym or encryption). The replacement of a span depends only on its type and its text, so redaction is a deterministic function of T and $\Pi(T)$ that acts on each span separately.

Definition 2 (Leak) Given a set G of gold sensitive spans for T, a *leak* is any character of a span in G that is not covered by a span in $\Pi(T)$. A gold span with at least one uncovered character is *leaked*, because a partly visible card number or key can often be reconstructed or narrowed.

Adversary and setting

The adversary is a later reader of the redacted output, for example the recipient of a shared log. The input may be larger than memory and arrive as an arbitrary sequence of chunks. The system runs locally and makes no network calls, so the unredacted text does not leave the host.

What is in scope

(i) That chunked and parallel processing introduce no leak beyond those of the whole-document pipeline; (ii) bounded working memory; (iii) the detection quality of the whole-document pipeline, measured empirically.

What is out of scope

Detection completeness: the pipeline misses values its detectors do not recognise, and the guarantee does not change that. Adversarial evasion: an author who encodes, splits or obfuscates a secret on purpose is not modelled. Inference from residual text: quasi-identifiers and context that remain after redaction may still allow re-identification [16, 23]. Side channels such as output length. Bounded memory is a resource property, not a security property.

4 Design

Figure 1 shows the architecture. The reference pipeline Π runs every configured detector on a text, adjusts confidences with a bounded contextual scorer, drops detections below a threshold (0.5 by default) and resolves overlaps into a disjoint set.

4.1 Detection tiers

Deterministic patterns with validators

Credential formats with a fixed shape (cloud access keys, provider tokens, JSON Web Tokens, PEM private-key blocks) and structured personal data (e-mail, telephone, card and social-security numbers, IPv4 and IPv6 addresses) are matched by regular expressions. A match becomes a detection only if it passes a validator: the Luhn checksum for card numbers [32], area, group and serial rules for social-security numbers, octet ranges for IPv4, address parsing for IPv6, and an E.164 shape check for telephone numbers. A generic key-assignment rule matches values that follow keys such as `api_key`, `token` or `password`.

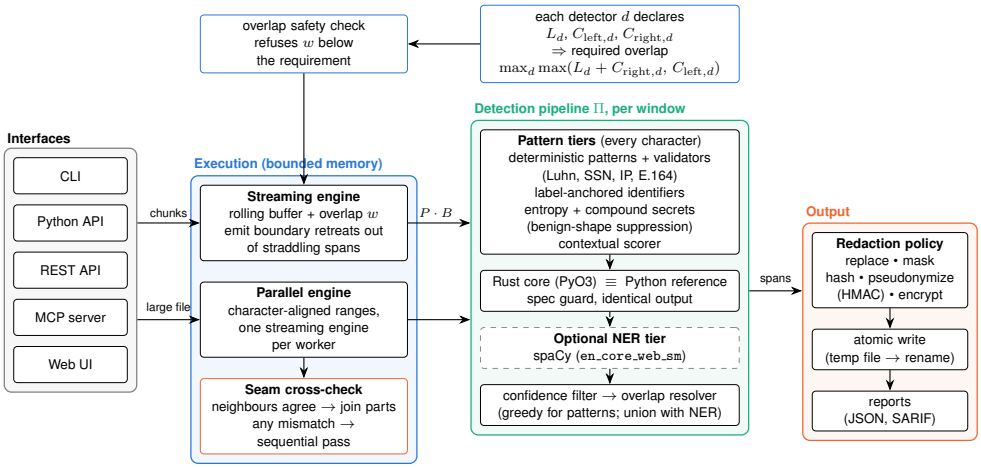


Fig. 1 System architecture. Interfaces feed text to the streaming engine or, for a large file, to the parallel engine, whose workers each run a streaming engine and whose seams are cross-checked. Each window passes through the detection pipeline Π: the pattern tiers run on every character, on the Rust core or the identical Python reference. The dashed named-entity tier (spaCy) is optional. The resolver produces disjoint spans, which the redaction policy rewrites and an atomic write commits. The overlap w is checked against the span and context bounds the active detectors declare.

Label-anchored identifiers

Unchecked numeric identifiers such as medical record, account or licence numbers are ambiguous in isolation, so this tier promotes a value to a detection only when a cue word that names it (for example `Patient ID:`) appears within 96 characters to its left.

Entropy tier with benign-shape suppression

Unstructured candidate runs of 20 to 8192 characters over a key alphabet are scored by length, character-class mix and Shannon entropy [28], with a lower threshold for hexadecimal runs; a compound variant covers runs with embedded symbols. Entropy alone cannot separate a secret from other random-looking strings. During this study every generated UUID, commit hash, SHA-256 digest and base64 image payload in the secrets development split was flagged (Section 7.4). The tier therefore suppresses three shapes: a canonical UUID, a 40- or 64-character hexadecimal value preceded by a digest cue (`commit`, `sha256`, `hash` and similar), and the payload of a `data:` URI. Suppression is vetoed when a secret-like key (`key`, `token`, `secret`, `password` and similar) names the value, so `SAUCE_ACCESS_KEY=`*uuid* remains a secret. The check reads at most 32 characters of left context.

Contextual scorer

A 48-character left window raises or lowers confidences when trigger words are present and emits a small set of trigger-gated detections of its own.

Algorithm 1 Streaming detection with overlap w

Require: chunks $c_1, c_2, \ldots$; pipeline Π; overlap w; minimum emit size $m \geq w$

$P \leftarrow \varepsilon$ ▷ emitted text kept as read-only left context
$B \leftarrow \varepsilon$ ▷ buffer of unemitted text
for each chunk c **do**
  $B \leftarrow B \cdot c$
  if $|B| < w + m$ **then**
    continue
  end if
  $S \leftarrow \{(i - |P|,\ j - |P|,\ \tau,\ \kappa) : (i, j, \tau, \kappa) \in \Pi(P \cdot B),\ i \geq |P|\}$
  $e \leftarrow |B| - w$ ▷ planned emit boundary
  while some $(i, j, \tau, \kappa) \in S$ has $i < e < j$ **do**
    $e \leftarrow i$ ▷ retreat out of the straddler
  end while
  if $e \leq 0$ **then**
    continue ▷ a span pins the buffer; read more
  end if
  emit $B[0{:}e]$ with $\{(i, j, \tau, \kappa) \in S : j \leq e\}$
  $P \leftarrow$ last w characters of $P \cdot B[0{:}e]$
  $B \leftarrow B[e{:}]$
end for
emit B with $\{(i - |P|,\ j - |P|,\ \tau,\ \kappa) : (i, j, \tau, \kappa) \in \Pi(P \cdot B),\ i \geq |P|\}$

Optional named-entity tier

For names, organisations and locations, an optional tier runs a spaCy pipeline (`en_core_web_sm` in our experiments).

4.2 Overlap resolution

Pattern detections that overlap are resolved best-first by confidence, then by length, so a card number beats a telephone number embedded in it. Named-entity detections are handled differently: any cluster of overlapping spans that contains a named-entity span becomes one span from the cluster's leftmost start to its rightmost end, labelled by its best pattern member if one exists. Without this union rule, a PERSON tag on `john.doe` could replace the e-mail detection for `john.doe@example.com` and leave the domain exposed.

4.3 Streaming engine

Algorithm 1, a simplified form of `engine.stream_windows` illustrated in Figure 2, keeps a buffer of unemitted text and the last w characters already emitted. Each pass detects over their concatenation, so detectors see real left context rather than an artificial edge, and emits only the part of the buffer that lies at least w characters before its end. If a detection crosses the planned emit boundary, the boundary retreats to the detection's start and the detection is carried into the next pass. Small chunks

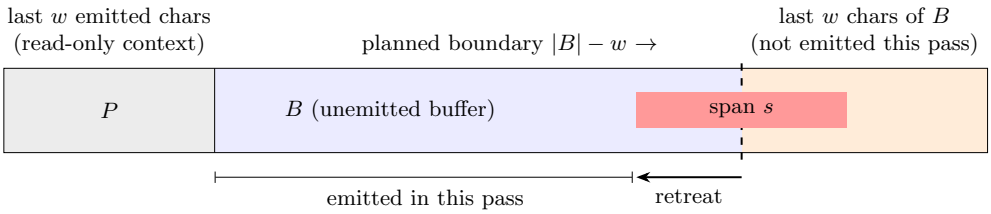


Fig. 2 One pass of Algorithm 1. The detector reads $P \cdot B$; only text before the planned boundary may be emitted, and the boundary retreats out of any span that crosses it, so s is carried to the next pass and detected whole.

are coalesced until at least m characters can be emitted, which bounds the cost of rescanning the overlap per emitted character regardless of how the input is chunked. The defaults are $w = 8192$ characters and $m = 65{,}536$ characters.

4.4 Parallel engine

For large files the parallel engine splits the input into character-aligned byte ranges of at least $4w$ characters, one per worker process. Each worker reads w characters of scanned padding and a further w characters of context-only prefix around its interior boundaries (seams). A span that crosses a seam belongs to the left worker, and the right worker begins emitting at that span's end. Neighbouring workers compute the seam position independently, and the coordinator compares their reports; on any disagreement it discards the parallel result and redacts the file sequentially.

4.5 Implementation

The pure-Python implementation is the reference. An optional Rust core implements the pattern tiers. At start-up it checks that its compiled pattern sources equal the Python ones, and it hands a window back to Python in the rare cases where Unicode case folding or floating-point summation could make the two disagree.

Use of AI tools

The large language model Claude (Anthropic) was used during this research to help write evaluation harness scripts, draft early versions of manuscript sections, format the LaTeX source for the journal template, refine prose and check bibliographic entries. The authors independently verified all experimental outputs, mathematical proofs and manuscript text, and take full responsibility for the final work.

5 Formal analysis

Let $S_w(T)$ be the set of detections emitted by Algorithm 1 for some chunking of T, mapped back to absolute offsets, and $P_{w,k}(T)$ the output of the parallel engine with k workers. Each detector $d \in \mathcal{D}$ declares three constants: the longest span it emits, L_d, and the context it reads to the left and right of a span, $C_{\text{left},d}$ and $C_{\text{right},d}$. The

two assumptions are stated per detector and must hold for every input text, not only for T, because the engine applies Π to windows.

Assumption A1 (Bounded span) For every text X, every detection of detector d in $\Pi(X)$ has length $j - i \leq L_d$.

Assumption A2 (Bounded context) Let X and Y be texts and δ an offset with $X[a{:}b] = Y[a{+}\delta : b{+}\delta]$. Let (i, j) be a span of detector d such that either $i - C_{\text{left},d} \geq a$ or $a = a + \delta = 0$, and either $j + C_{\text{right},d} \leq b$ or both $b = |X|$ and $b + \delta = |Y|$. Then $(i, j, \tau, \kappa) \in \Pi(X)$ if and only if $(i + \delta, j + \delta, \tau, \kappa) \in \Pi(Y)$.

In words, A2 says that whether a span of d is in the final detection set, and with which type and confidence, depends only on the $C_{\text{left},d}$ characters before it, the span itself and the $C_{\text{right},d}$ characters after it, where a window edge that coincides with an edge of both texts counts as real. The pipeline requirement is

$$w^\star \;=\; \max_{d \in \mathcal{D}} \; \max\big(L_d + C_{\text{right},d},\; C_{\text{left},d}\big). \tag{1}$$

Theorem 1 (Streaming equivalence) *Under A1 and A2, if $w \geq w^\star$ and $m \geq w$, then $S_w(T) = \Pi(T)$ for every text T and every chunking of T.*

Corollary 1 (Pipeline-wide sufficient condition) *Let $L_{\max} = \max_d L_d$, $C_{\text{left}} = \max_d C_{\text{left},d}$ and $C_{\text{right}} = \max_d C_{\text{right},d}$. Then $w \geq \max(L_{\max} + C_{\text{right}}, C_{\text{left}})$ implies $w \geq w^\star$, so Theorem 1 applies.*

The corollary is the intuition: the overlap must hold the longest span together with the widest right context, and the widest left context. It is sufficient but not necessary, because it pairs the longest span with a right context that may belong to a different detector. Equation (1) is the condition the engine checks (Section 5.1).

Why the right context is added to the span

An overlap of $\max_d \max(L_d, C_{\text{left},d}, C_{\text{right},d})$, which bounds each quantity separately, is not enough. Take a detector d that rejects a candidate whose right context is missing, and an overlap $w < L_d + C_{\text{right},d} - 1$. A chunking can end a pass so that a span $s = (i, j)$ of $\Pi(T)$ starts one character before the planned boundary $e_0 = |B| - w$ and has length L_d. Its right context then extends $(L_d - 1) + C_{\text{right},d} > w$ characters beyond e_0, past the end of the buffer. The window therefore rejects s. If no other detection straddles e_0, the boundary does not retreat, and the pass emits $T[i : e_0]$, the first character of s, unredacted. A shorter span ending before e_0 can be emitted in its place. The bound in Eq. (1) is thus tight up to one character. It is also why the safe-region step in the proof below adds $C_{\text{right},d}$ to L_d.

Proof of Theorem 1 Fix a non-final pass. Let p be the absolute position of $B[0]$, $q = p + |B|$ and $e_0 = q - w$ the planned boundary. The window is $W = T[p - |P| : q]$, where $|P| = \min(w, p)$ because P holds the last w emitted characters.

Safe region. Let $s = (i, j, \tau, \kappa)$ be a span of detector d, in absolute offsets, with $p \le i < e_0$ that belongs to $\Pi(W)$ or to $\Pi(T)$. By A1, $j \le i + L_d < q - w + L_d \le q - C_{\text{right},d}$, using $w \ge L_d + C_{\text{right},d}$. On the left, $i - C_{\text{left},d} \ge p - w$ because $w \ge C_{\text{left},d}$; if $p < w$, the window starts at position 0 of T. A2, applied to $X = W$ and $Y = T$, then gives $s \in \Pi(W) \Leftrightarrow s \in \Pi(T)$. So the spans of the window that start in $[p, e_0)$ are exactly the spans of $\Pi(T)$ that start there.

Retreat. $\Pi(W)$ is pairwise disjoint, so at most one span s^* straddles e_0, and it starts before e_0, so it belongs to $\Pi(T)$. The boundary becomes $e = i^*$ if s^* exists and $e = e_0$ otherwise. In the first case, $i^* > e_0 - L_d \ge p + m - L_d \ge p$, because $|B| \ge w + m$ and $m \ge w \ge L_d$. So $e > p$, and the pass emits at least one character; the pinned branch is never taken. No span of $\Pi(T)$ straddles e. If $e = e_0$, such a span would start before e_0 and so be the straddler of W. If $e = i^*$, it would overlap s^*, which disjointness rules out.

Emission. The pass emits $T[p{:}e]$ with the window spans contained in it. By the safe region these are exactly the spans of $\Pi(T)$ contained in $[p, e)$. The final pass has $q = |T|$, so the right-hand clause of A2 holds for every span, and it emits exactly the spans of $\Pi(T)$ inside $[p, |T|)$. The emitted segments partition T, and no span of $\Pi(T)$ crosses a segment boundary. Every span of $\Pi(T)$ is therefore emitted exactly once, with the type and confidence it has in $\Pi(T)$, and nothing else is emitted. □

Theorem 2 (Parallel identity with fail-safe fallback) *(i) Under the conditions of Theorem 1, and with ranges of at least $4w$ characters, every seam report agrees and $P_{w,k}(T)$ equals the sequential output byte for byte. (ii) Without any assumption on the detectors, the output is either assembled from parts whose seam reports all agree, so that every character of T is emitted exactly once, or it is the sequential output.*

Theorem 3 (Bounded working buffer) *Let c be the length of the longest input chunk. Under the conditions of Theorem 1, the unemitted buffer of S_w never exceeds $w + m + c$ characters, and each detection window never exceeds $2w + m + c$, independent of $|T|$. Each parallel worker additionally holds back at most $2w$ characters plus one segment, so the parallel engine needs $O\big(k\,(w + m + c)\big)$ characters.*

Because redaction acts on each span separately (Definition 1), Theorem 1 implies byte-identical redacted output. The proofs of Theorems 2 and 3, and a table mapping each step to the implementation, are in Appendix A. For `redact_file`, $c = m = 65{,}536$ and $w = 8192$, so the buffer stays below 139,264 characters and each window below 147,456 characters. The proofs were not machine-checked.

5.1 What is derived and what is assumed

Each detector declares $(L_d, C_{\text{left},d}, C_{\text{right},d})$. The engine computes $w^\star$ from Eq. (1) over the active detectors and refuses to start with a smaller overlap unless the caller explicitly overrides it. Table 1 lists the declarations. The detectors that emit the longest spans (private-key blocks and entropy candidates, $L_d = 8192$) read no right context. The only detector with right context among the default tiers, the label-anchored tier, emits spans of at most 64 characters. Hence $w^\star = 8192$, and the default overlap $w = 8192$ satisfies Theorem 1 exactly, with or without the spaCy tier. The pipeline-wide Corollary 1 would ask for $8192 + 8 = 8200$ characters; that is why the per-detector form is the one the engine enforces. A regression test fails if any

Table 1 Declared streaming bounds of the evaluated detectors, in characters, at the evaluated commit. The requirement of a detector is $\max(L_d + C_{\text{right},d}, C_{\text{left},d})$; the pipeline requirement $w^\star$ is the largest requirement among active detectors.

Detector d	L_d	$C_{\text{left},d}$	$C_{\text{right},d}$	Requirement
Deterministic patterns	8192	0	0	8192
Label-anchored identifiers	64	96	8	96
Entropy tier	8192	32	0	8192
Compound entropy tier	8192	0	0	8192
Contextual scorer	9	48	0	48
spaCy named-entity tier (optional)	256	256	256	512
Default pipeline, $w^\star$				8192
Default pipeline with spaCy, $w^\star$				8192

declaration changes, and a differential test probes each detector to check that its real context dependency does not exceed its declaration.

Two parts of A1 and A2 are *not* derived from declarations. They should be read as assumptions corroborated by tests rather than proven properties of the code. (i) The confidence adjustment and threshold must not reach beyond the declared context of the detector whose span they adjust. The contextual scorer's 48-character left window is within w for every detector, but it is not added to the other detectors' declarations. (ii) Overlap resolution must not propagate decisions along chains of overlapping candidates beyond the declared contexts, and the named-entity union rule must not create spans longer than the declared bounds. For the pattern tiers, whose spans are bounded and whose overlaps are short, the equivalence tests in Section 7.1 found no counterexample. A statistical model's output can depend on its whole input window, so for the spaCy tier the declared bounds of 256 characters are declarations only, and the theorems should not be read as covering it. Empirically, its streamed output matched the whole-document output in every chunking we tried (Section 7.1).

Failure modes

Lowering the overlap below $w^\star$ is refused by default. If a caller overrides that, the parallel engine still cannot lose or duplicate characters (Theorem 2(ii)), but the sequential engine can split a long span. A detector that understates its context, or one that needs document-wide context, is outside the guarantee.

6 Experimental methodology

The evaluation addresses six questions. **RQ1**: do the implementation's streamed, parallel and native outputs match the reference in practice, and how often does chunking without the guarantee leak? **RQ2**: how do memory and throughput scale? **RQ3**: how well do the pattern tiers detect credentials compared with purpose-built scanners? **RQ4**: how well does SafeStream detect personal data compared with Presidio, with and without a named-entity model? **RQ5**: how much protection does it give at the level of a whole document, and how much of that depends on rules written after seeing the benchmark? **RQ6**: how well do the systems detect personal data in real system logs?

Table 2 Test splits. Gold-free documents contain no gold span; any detection on them is a false alarm. Clusters are the resampling units of the cluster and two-stage bootstrap (Section 6.4).

Corpus	Documents	Gold spans	Gold-free	Bytes	Clusters
presidio-research	750	1,394	63	63,678	200 templates
secrets	313	293	97	46,595	96 rules + 5 templates
RedactionBench	100	25,592	0	1,378,267	11 domains
OpenSSH logs (real)	2,000	2,491	148	221,218	27 templates

6.1 Corpora and splits

Table 2 lists the test splits. Each corpus was split once into development and test halves with a fixed seed. The manifests list document identifiers and the SHA-256 of the source data, and the loader refuses a source that no longer matches. Development, error analysis and tuning used development halves only, with the exception of RedactionBench described below; test halves were scored after the code was frozen, with one disclosed exception (Section 7.4).

- **presidio-research** [15] (synthetic): the dataset distributed with Presidio's evaluation toolkit (revision 0cb3650), with person, location, organisation, telephone, card, date and other labels, generated from templates. The 750 test documents come from 200 templates (median 3.5 documents per template).
- **secrets** (semi-synthetic): the labelled examples in the Nosey Parker rule set [29] (revision 2e6e7f3) as positives, with gold spans given by each rule's own capture groups, plus 200 generated non-secrets (UUIDs, commit hashes, SHA-256 digests, base64 image payloads, minified JavaScript) as gold-free documents. The split is by rule, so the test half contains vendor formats absent from the development half; its 216 positives come from 96 rules. The positives are short examples written by rule authors, not credentials observed in real artefacts.
- **RedactionBench** [33]: 200 documents across eleven domains, a majority seeded from real-world sources according to its authors, with character spans labelled *mandatory* or *contextual*. The test half holds between 5 and 17 documents per domain. All 200 documents were used for error analysis and rule writing before the split existed, so its test half is not held out (Section 7.5).
- **OpenSSH logs** (real): the 2,000-line OpenSSH sample of Loghub [34], real `sshd` lines from a server under attack, one document per line. Gold comes from labels on the 27 log templates that Loghub's parser assigns: the value of a slot that names a remote host is `IP_ADDRESS` when it is a valid IP address (reverse-DNS hostnames are excluded), and the value of a slot that names an account, and any account name written into a template, is `USERNAME`. Applying the labels gives 1,349 IP addresses and 1,142 user names; 148 lines contain neither. The labels were drafted from the template text with an AI assistant and checked by the first author, and the corpus was frozen as test-only (commit `099cc9c`) before any system was run on it. No SafeStream rule was written with these logs in view.

6.2 Systems and configurations

SafeStream was run with the Rust core and default tiers (*pattern*), with the opt-in date tier (*+dates*), and with the spaCy `en_core_web_sm` tier (*+spaCy*). **Presidio** 2.2.355 [4] was run as its pattern recognisers alone (*regex*) and as the full analyser with spaCy `en_core_web_sm` (*sm*) and `en_core_web_lg` (*lg*), with spaCy 3.8.13 and models 3.8.0. **gitleaks** 8.28.0 [5] and **TruffleHog** 3.90.8 [6] were run with default rules and default false-positive filtering; TruffleHog ran with `--no-verification`, so no example credential was sent to a live service. A sensitivity run repeated both scanners with their false-positive filters disabled (Section 7.3). Each scanner ran once over a directory with one file per document, and findings were mapped back to character offsets; TruffleHog's composite findings (for example `user:token`) were credited for each component that appears in the text. Presidio received documents longer than 20,000 characters in line-aligned pieces of at most that length; no gold span in any test split crosses a piece boundary, so the splitting cost Presidio no recall. Each Python tool ran in its own virtual environment and process, pinned to one thread; the scanners are Go binaries and ran with their default concurrency. All SafeStream thresholds are defaults; no threshold was tuned on test data. The RedactionBench dev-selected configurations (Section 7.5) were chosen on the development half by a rule fixed in advance.

6.3 Metrics

Let G and R be the sets of gold and redacted character positions in a document. Character precision and recall are $|G \cap R|/|R|$ and $|G \cap R|/|G|$, summed over documents, and F1 is their harmonic mean. All metrics are type-agnostic: a value counts as protected whatever label it receives. The *fully redacted rate* is the share of gold spans with every character redacted. The *false-alarm document rate* is the share of gold-free documents with at least one detection. On RedactionBench, *mandatory recall* is character recall over mandatory spans, and the *no-leak document rate* is the share of documents in which every mandatory character is redacted (Section 7.5 discusses how it relates to recall).

6.4 Statistical analysis

We report 95% percentile bootstrap intervals with 2,000 resamples and seed 0 [35]. All systems evaluated on a corpus are scored on one shared resample matrix, so every difference between two systems is paired; we report the median paired difference and call it significant only when its paired interval excludes zero. Documents are not independent. Secrets positives cluster by rule, presidio-research documents share generator templates, and RedactionBench documents share domains. We therefore use three resampling schemes on the same per-document counts [36]: *document* (documents drawn independently), *cluster* (whole clusters drawn with replacement, all their documents kept) and *two-stage* (clusters drawn with replacement, then documents drawn with replacement inside each drawn cluster). Table 2 lists the clusters. Tables 5, 7 and 10 give document-level intervals. Table 8 repeats the planned comparisons under all three schemes, and we call a difference *robust* when it is significant under all three.

RedactionBench has only eleven clusters, so its cluster intervals are themselves imprecise; we read them as a sensitivity analysis rather than as an exact test. No correction for multiple comparisons was applied; the comparisons in the abstract were planned, the ablation comparisons were not. The analysis was not pre-registered.

6.5 Environment

Accuracy runs used a 4-vCPU Intel Xeon virtual machine at 2.80 GHz with 15 GB RAM, Linux and CPython 3.13. Throughput figures come from separate runs, five per configuration, on the same machine with no other benchmark running; the throughput recorded during accuracy runs is not used. Memory was measured as the peak resident set size of a fresh process per point minus its resident set size after imports, so native allocations are included; it is reported in MiB. The cluster analysis re-ran every test-split system at the evaluated commits with the same tool versions and saved per-document predictions; each point estimate and each document-level paired interval reproduced the published values to three decimals. The held-out results come from commit `0765dae`, the results after the entropy fix (Section 7.4) from `3d2c6ac`, and the pre-exposure RedactionBench results from `874e105`, all on the main branch of the public repository.

7 Results

7.1 RQ1: equivalence, backend parity and leaks caused by chunking

On every test split the Rust core and the pure-Python reference produced identical predictions (presidio-research 178 spans, secrets 298, RedactionBench 3,167, from the frozen commit). The repository's equivalence suite asserts byte-identical streamed, whole-document and parallel output over exhaustive small chunkings, property-based generated inputs, seam-straddling entities and forced fallbacks, and also checks native parity and declared context; it passed 133 of 133 tests at the final commit. These tests corroborate the theorems for the configurations they exercise; they are not proofs, and they do not exercise the named-entity tier's full input dependence.

How often chunking alone leaks

The theorems say that chunking cannot add leaks; they do not say how many leaks chunking adds without them. To measure that, we concatenated each test split into one stream, two newlines between documents, and ran the same pattern pipeline in four ways (Table 3; script `benchmarks/chunking_leak.py`):

- *whole*: one pass over the stream, the reference $\Pi(T)$;
- *naive*: fixed chunks of c characters, each detected on its own;
- *overlapping windows*: windows of $c + o$ characters advancing by c, each keeping the detections that start in its own c characters, which is common practice; we used $o = 256$ and $o = 8192$, the longest declared span;

- *SafeStream*: Algorithm 1 at its default overlap and the smallest allowed emit size ($m = w$), on five random chunkings per chunk size with chunk lengths drawn uniformly from 1 to $2c$.

We also ran a 4.2 MB log produced by the repository's generator, with 56,153 gold values, as a stand-in for a dense machine log, and the real system logs of Loghub [34]: its 2,000-line samples of 16 systems (SSH, Linux, Apache, Hadoop, HDFS, OpenStack and others; 4.4 MB, repository commit `dd61d09`). Loghub carries no redaction labels, so on it we count only missed reference detections.

Naive chunking leaked gold values that the whole-document pass redacts on every corpus. At $c = 1$ KiB it leaked 237 RedactionBench values, 7 on presidio-research and 5 on secrets, and 872 values in the generated log; at 64 KiB it still leaked 1 RedactionBench value and 12 in the log. A leaked value can be part of a larger gold region, so the number of leaked gold spans can exceed the number of missed detections. Overlapping windows removed most of these leaks, but not all. At 1 KiB they still missed 3 RedactionBench detections and leaked 2 RedactionBench values, 1 presidio-research value and 2 secrets, even with $o = 8192$. In the pattern runs with $o = 8192$, all seven residual misses start 5 to 9 characters into their window and come from detectors that read a cue to their left (a key name, a URL-credential prefix, an account label). The cue lay in the previous window, which these windows discard, so a longer overlap o, which only extends a window to the right, cannot recover them. SafeStream keeps exactly this context as its read-only prefix P. Both naive and overlapping chunking also redacted characters the whole-document pass does not (up to 2,738 in the log), from candidates cut short at a window edge. The real logs behaved like the test corpora: on Loghub, naive chunking missed 388 of the 18,329 reference detections at 1 KiB and 9 at 64 KiB, and overlapping windows still missed 3 at 1 KiB and 1 at 4 KiB. SafeStream's detection set was identical to the whole-document reference in all 100 pattern-tier runs: five streams, four chunk sizes and five chunkings each. The same held with the spaCy tier on presidio-research, on the first 66 RedactionBench test documents (928,652 characters, below spaCy's 1,000,000-character input limit) and on the first three Loghub systems (761,472 characters), at 4 KiB and 64 KiB with two chunkings each. That last result is an empirical check, not a proof, because the theorems do not cover the spaCy tier (Section 5.1).

7.2 RQ2: memory and throughput

Figure 3 plots peak resident memory against input size. Each point is a fresh process redacting a generated log file dense with personal data and credentials. With the Rust core, streaming `redact_file` added between 8.4 and 8.54 MiB at every size from 8 MiB to 2 GiB, a 256-fold range. The pure-Python backend added between 4.3 and 4.4 MiB from 8 to 512 MiB; it was not run at larger sizes because of its run time (726 s at 512 MiB). Whole-document `redact()` on the same generated inputs, measured the same way, added 235.5 MiB at 8 MiB and 904.6 MiB at 32 MiB, about 28 times the input size, because the input string, the detections and the output are held at once; at 32 MiB the two modes therefore differ by a factor of about 106 on identical input. A 128 MiB whole-document run was stopped after 20 minutes without completing and

Table 3 Leaks caused by chunking alone. Each cell gives missed reference detections / gold spans leaked by chunking (fully redacted by the whole-document pass, not by the variant); Loghub has no gold, so its cells give missed detections only. SafeStream: runs whose detection set is identical to the whole-document reference, out of the random chunkings tried. Overlapping windows keep the detections that start in their own chunk; o is the window overlap in characters. The generated log is synthetic and dense (56,153 gold values in 4.2 MB).

Corpus (pipeline)	Chunk c	Naive	$o = 256$	$o = 8192$	SafeStream
secrets (pattern)	1 KiB	11 / 5	4 / 3	2 / 2	5 of 5
	4 KiB	3 / 1	1 / 1	0 / 0	5 of 5
	64 KiB	0 / 0	0 / 0	0 / 0	5 of 5
presidio-research (pattern)	1 KiB	5 / 7	1 / 1	1 / 1	5 of 5
	4 KiB	0 / 0	0 / 0	0 / 0	5 of 5
	64 KiB	0 / 0	0 / 0	0 / 0	5 of 5
RedactionBench (pattern)	1 KiB	74 / 237	3 / 2	3 / 2	5 of 5
	4 KiB	9 / 10	1 / 0	1 / 0	5 of 5
	64 KiB	1 / 1	0 / 0	0 / 0	5 of 5
Generated log (pattern)	1 KiB	957 / 872	0 / 0	0 / 0	5 of 5
	4 KiB	234 / 212	0 / 0	0 / 0	5 of 5
	64 KiB	16 / 12	0 / 0	0 / 0	5 of 5
presidio-research (+spaCy)	4 KiB	2 / 1	0 / 0	0 / 0	2 of 2
	64 KiB	0 / 0	0 / 0	0 / 0	2 of 2
RedactionBench, 66 docs (+spaCy)	4 KiB	20 / 10	2 / 0	2 / 0	2 of 2
	64 KiB	0 / 0	0 / 0	0 / 0	2 of 2
Loghub, 16 real systems (pattern)	1 KiB	388	3	3	5 of 5
	4 KiB	94	1	1	5 of 5
	64 KiB	9	0	0	5 of 5
Loghub, 3 systems (+spaCy)	4 KiB	18	4	4	2 of 2
	64 KiB	1	0	0	2 of 2

The 16 KiB rows are in the result files (`research/results/raw/2026-11-12_chunking_leak_*.json`); they fall between the 4 KiB and 64 KiB rows.

Table 4 Throughput on the test splits in MB/s, mean ± sample standard deviation over 5 runs, one thread, load time excluded. Each run is a fresh process on an otherwise idle 4-vCPU machine.

System	presidio-research	secrets	RedactionBench
SafeStream pattern (Rust core)	1.82 ± 0.42	2.39 ± 0.58	10.4 ± 1.3
SafeStream pattern (pure Python)	1.10 ± 0.07	0.916 ± 0.17	1.23 ± 0.15
SafeStream +spaCy (sm)	0.0118 ± 0.0009	0.0227 ± 0.0034	0.0467 ± 0.0033
Presidio regex	0.0802 ± 0.0127	0.0983 ± 0.0155	0.0694 ± 0.0028
Presidio sm	0.0081 ± 0.0004	0.0125 ± 0.0007	0.0201 ± 0.0003
Presidio lg	0.0074 ± 0.0010	0.0128 ± 0.0008	0.0203 ± 0.0003
gitleaks[1]	0.507 ± 0.028	0.409 ± 0.051	3.15 ± 0.19
TruffleHog[1]	0.0269 ± 0.0016	0.0188 ± 0.0007	0.594 ± 0.020

[1] Go binaries run once per corpus over a directory of documents; the figure includes process start-up and is not limited to one thread.

is not reported. These measurements are consistent with Theorem 3 but cover one generated input shape on one machine, and only two whole-document sizes.

Table 4 reports throughput on the test splits. Per-document overhead dominates on presidio-research and secrets, whose documents average 85 and 149 bytes; RedactionBench's documents average 13.8 kB. There the Rust core processed 10.4 ± 1.3 MB/s, the pure-Python reference 1.23±0.15 MB/s and Presidio's pattern recognisers 0.069 MB/s. With a statistical model in the loop every system ran below 0.05 MB/s: SafeStream +spaCy at 0.047 MB/s and Presidio sm and lg at 0.020 MB/s. The scanner figures include process start-up and are not comparable per thread.

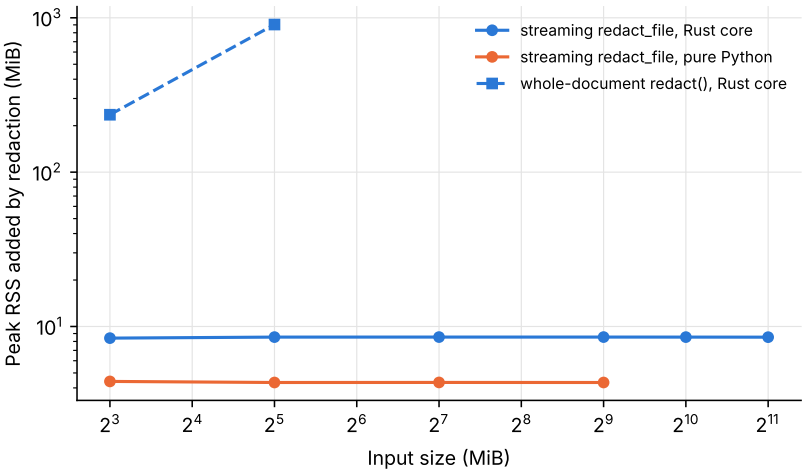


Fig. 3 Peak resident memory added by redaction (peak RSS minus RSS after imports, MiB) against input size, log scales, each point a fresh process. Streaming `redact_file` stays flat from 8 MiB to 2 GiB (Rust core) and to 512 MiB (pure Python); whole-document `redact()` grows with the input.

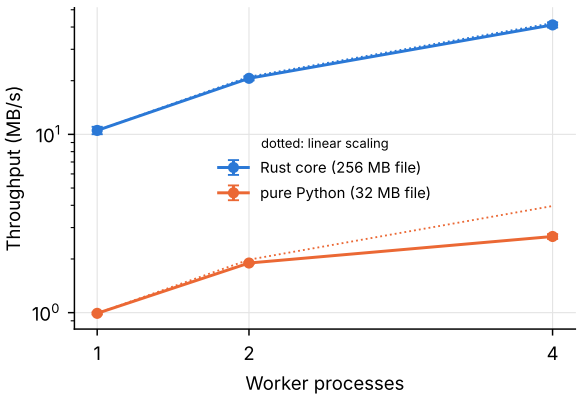


Fig. 4 Throughput of `redact_file` against worker processes on a 4-vCPU machine, mean and standard deviation over 5 runs (log scale). Dotted lines show linear scaling from one worker. The parallel output is byte-identical to the sequential output (Theorem 2).

On a single 256 MB generated log file (mostly benign log lines and prose with personal data and credentials mixed in), `redact_file` with the Rust core ran at 10.5 ± 0.5, 20.6 ± 0.5 and 41.1 ± 1.4 MB/s with 1, 2 and 4 worker processes, a 3.9-fold speed-up on 4 vCPUs (Figure 4). The pure-Python backend on a 32 MB file of the same kind ran at 0.99, 1.90 and 2.67 MB/s. These figures come from one virtual machine and should be read as relative rather than absolute.

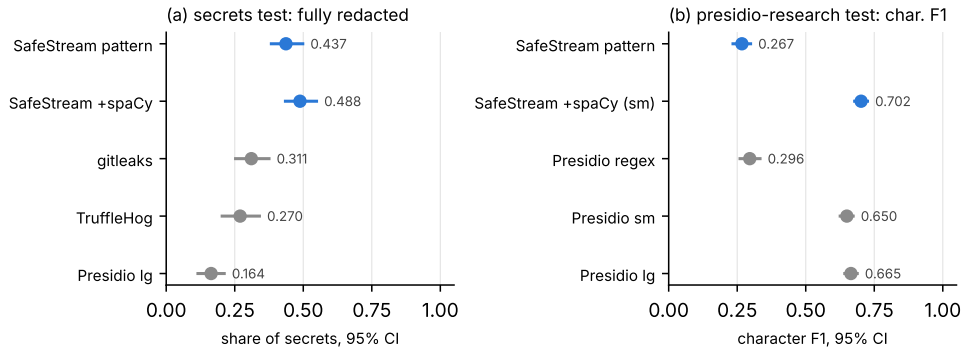


Fig. 5 Test-split accuracy with 95% document-level bootstrap intervals. (a) Share of secrets fully redacted on the secrets test split. (b) Character F1 on the presidio-research test split. SafeStream configurations in blue, baselines in grey.

7.3 RQ3 and RQ4: detection on held-out data

Table 5 and Figure 5 give the test-split results; Table 8 tests the planned differences under clustering.

Credentials (RQ3)

On secrets, SafeStream's pattern tiers fully redacted 0.437 of test secrets, against 0.311 for gitleaks and 0.270 for TruffleHog. With documents resampled independently, the paired differences in fully redacted rate were +0.128 [+0.073, +0.183] and +0.167 [+0.079, +0.250], and in character F1 +0.233 [+0.043, +0.441] and +0.536 [+0.310, +0.685]. The differences in character precision were not significant (+0.005 [−0.063, +0.123] and +0.060 [−0.109, +0.176]). Resampling by rule widens every interval (Table 8). The advantage over gitleaks remains significant in both fully redacted rate (two-stage +0.125 [+0.025, +0.209]) and character F1 (+0.226 [+0.011, +0.538]). Against TruffleHog the character-F1 advantage remains (+0.530 [+0.192, +0.742]), but the fully-redacted-rate advantage does not (+0.184 [−0.008, +0.363]); that margin is not robust to dependence between examples of the same rule. SafeStream flagged one of 97 generated non-secrets; neither scanner flagged any. More than half of the test secrets (0.563) were not fully redacted by SafeStream either. The development recall (0.542) exceeded the test recall (0.437) because the test half holds vendor formats for which SafeStream has no pattern. Presidio has no credential recognisers, and adding spaCy raised SafeStream's false-alarm rate to 0.340; on the development split those false alarms were the word `Merge` in commit lines tagged as a person and digest or UUID tokens tagged as persons or organisations.

Scanner sensitivity

The gold labels are Nosey Parker's rule examples, and both scanners apply false-positive filters designed to suppress example- or placeholder-like strings, so their default configurations may be penalised more than they would be on real data. We

Table 5 Test-split results with 95% document-level bootstrap intervals. SafeStream rows are after the entropy fix of Section 7.4, which changed only the secrets results. FR: fully redacted rate; FA: false-alarm document rate.

Corpus / system	Char P	Char R	Char F1	FR	FA
presidio-research (750 documents)					
SafeStream pattern	.995 [.992, .998]	.154 [.133, .176]	.267 [.235, .299]	.115 [.098, .134]	.000
SafeStream +dates	.996 [.993, .998]	.169 [.146, .192]	.290 [.255, .322]	.129 [.110, .150]	.000
SafeStream +spaCy (sm)	.898 [.877, .918]	.576 [.547, .606]	.702 [.679, .725]	.538 [.509, .568]	.079
Presidio regex	.948 [.919, .974]	.175 [.151, .202]	.296 [.260, .333]	.123 [.105, .145]	.000
Presidio sm	.871 [.848, .893]	.518 [.491, .546]	.650 [.626, .673]	.506 [.479, .536]	.048
Presidio lg	.878 [.856, .900]	.535 [.507, .563]	.665 [.642, .688]	.562 [.534, .590]	.000
secrets (313 documents)					
SafeStream pattern	.857 [.705, .946]	.829 [.740, .887]	.842 [.736, .906]	.437 [.384, .498]	.010
SafeStream +spaCy (sm)	.757 [.604, .858]	.845 [.762, .897]	.799 [.683, .870]	.488 [.435, .548]	.340
gitleaks	.837 [.630, .962]	.473 [.299, .679]	.604 [.430, .765]	.311 [.253, .375]	.000
TruffleHog	.803 [.719, .878]	.189 [.117, .308]	.306 [.204, .446]	.270 [.204, .340]	.000
Presidio regex	.282 [.211, .356]	.105 [.063, .177]	.153 [.098, .225]	.126 [.083, .168]	.392
Presidio sm	.251 [.191, .315]	.119 [.072, .197]	.161 [.107, .232]	.160 [.112, .206]	.649
Presidio lg	.308 [.238, .381]	.131 [.082, .214]	.184 [.124, .262]	.164 [.116, .212]	.433

therefore re-ran both scanners with these filters disabled (Table 6). For gitleaks we removed the global allowlist and all nine per-rule allowlists, which hold its stopwords, from the release's own configuration file, leaving all 217 rules (regexes, keywords and entropy thresholds) unchanged; the unmodified file reproduced the built-in default exactly. For TruffleHog we added the results its false-positive filter drops (`--results` including `filtered_unverified`), still without verification. Both filters-off runs are supersets of the defaults. gitleaks gained substantially in character recall (0.473 to 0.737) and F1 (0.604 to 0.805) but little in fully redacted rate (0.311 to 0.328): its filters suppressed few findings, in 6 of 313 documents, but long ones. Against filters-off gitleaks, SafeStream's fully-redacted-rate advantage remains robust (+0.111 [+0.058, +0.163]; two-stage +0.106 [+0.008, +0.193]), whereas its character-F1 advantage shrinks to +0.039 [+0.008, +0.093] and does not survive rule clustering (+0.039 [−0.007, +0.141]). TruffleHog changed little (fully redacted 0.270 to 0.290, F1 0.306 to 0.314), and the conclusions against it are unchanged: the F1 advantage is robust and the fully-redacted-rate advantage is not. On RedactionBench, disabling the filters raised gitleaks' mandatory recall from 0.247 to 0.284 and left TruffleHog essentially unchanged; pre-exposure SafeStream exceeded both filters-off scanners in character F1 and mandatory recall under all three schemes. SafeStream's own credential patterns were not developed on the secrets corpus: it was added after the split was fixed, and the only change made using it was the suppression rule of Section 7.4, designed on the development half.

Table 6 Secret scanners on the secrets test split with default false-positive filters and with the filters disabled, with 95% document-level intervals, and SafeStream's pattern tiers minus each scanner (paired, two-stage rule-clustered bootstrap). FR: fully redacted rate. † marks an interval that includes zero.

Scanner	Filters	Char F1	FR	SafeStream − scanner, F1	SafeStream − scanner, FR
gitleaks	default	.604 [.430, .765]	.311 [.253, .375]	+.226 [+.011, +.538]	+.125 [+.025, +.209]
gitleaks	disabled	.805 [.664, .885]	.328 [.269, .394]	+.039† [−.007, +.141]	+.106 [+.008, +.193]
TruffleHog	default	.306 [.204, .446]	.270 [.204, .340]	+.530 [+.192, +.742]	+.184† [−.008, +.363]
TruffleHog	disabled	.314 [.211, .454]	.290 [.225, .363]	+.522 [+.182, +.737]	+.163† [−.028, +.358]

Personal data (RQ4)

On presidio-research, the pattern tiers were precise (character precision 0.995) but recalled only 0.154 of sensitive characters, because most gold spans are names, locations and organisations that patterns cannot capture. Against Presidio's pattern recognisers, the paired difference in character recall was −0.021 [−0.041, −0.002] and in precision +0.047 [+0.021, +0.076]; under template clustering the precision advantage remains (+0.046 [+0.011, +0.103]) and the recall deficit is no longer significant. With the same spaCy model, SafeStream +spaCy exceeded Presidio sm in character F1 by +0.051 [+0.027, +0.076] and in fully redacted rate by +0.031 [+0.004, +0.060]. The F1 advantage is robust to template clustering (two-stage +0.052 [+0.013, +0.093]); the fully-redacted-rate advantage is not (+0.033 [−0.014, +0.080]). Against Presidio lg, which uses a larger model, SafeStream +spaCy had higher character F1 with documents resampled independently (+0.036 [+0.011, +0.064]), but not under template clustering (+0.037 [−0.003, +0.080]). The difference in fully redacted rate, −0.025 [−0.053, +0.004], was not significant under any scheme. This corpus is synthetic and template-generated; the comparison shows relative behaviour on that distribution only.

7.4 False alarms on high-entropy non-secrets

Before the fix described in Section 4.1, SafeStream flagged 83 of the 103 generated non-secrets in the development split (false-alarm rate 0.806), all through the entropy tier: every UUID, commit hash, SHA-256 digest and base64 image payload, and none of the minified JavaScript. The suppression rule was designed on the development split. A first version cost one development secret, a UUID-shaped access key assigned to `SAUCE_ACCESS_KEY`, which led to the secret-key veto. With the veto, the development false-alarm rate fell to 0.000 with no change in recall, and the presidio-research and RedactionBench development results were unchanged except in the fourth decimal. On the test split, the false-alarm rate fell from 0.794 [0.714, 0.872] to 0.010 [0.000, 0.035], character precision rose from 0.589 to 0.857, and recall was unchanged (0.829 character recall, 0.437 fully redacted). The test half had been scored once with the pre-fix code before the fix was made; both scores are reported. Because the gold-free documents in both halves come from the same generator, the test improvement is an optimistic estimate of the fix's effect on real non-secrets.

Table 7 RedactionBench test half (100 documents), with 95% document-level bootstrap intervals. *Exposed*: current code, whose rules were written after inspecting all 200 documents. *Pre-exposure*: the last commit before the benchmark entered the repository. *Dev-selected*: configuration chosen on the development half by a fixed rule (maximum development character F1) and scored on the test half once.

System	Char F1	Mandatory recall	No-leak docs
Pre-exposure, pattern	.518 [.434, .593]	.589 [.476, .696]	.074 [.022, .129]
Pre-exposure, +spaCy	.472 [.395, .547]	.674 [.566, .779]	.137 [.073, .208]
Pre-exposure, dev-selected	.469 [.394, .543]	.676 [.568, .781]	.137 [.073, .208]
Exposed, pattern	.579 [.490, .662]	.682 [.578, .787]	.084 [.032, .144]
Exposed, +spaCy	.512 [.429, .590]	.761 [.657, .860]	.179 [.105, .263]
Exposed, dev-selected	.508 [.427, .588]	.762 [.658, .862]	.179 [.105, .263]
gitleaks	.214 [.130, .314]	.247 [.140, .372]	.021 [.000, .053]
TruffleHog	.030 [.017, .048]	.024 [.013, .040]	.000
Presidio regex	.323 [.250, .393]	.331 [.227, .422]	.011 [.000, .033]
Presidio sm	.336 [.265, .402]	.416 [.326, .500]	.084 [.032, .146]
Presidio lg	.344 [.277, .409]	.420 [.333, .499]	.105 [.052, .174]

7.5 RQ5: document-level protection and contamination

Table 7 separates the contaminated and uncontaminated results. Rules written after inspecting the benchmark raised test mandatory recall of the pattern tiers by +0.091 [+0.044, +0.163] and character F1 by +0.060 [+0.029, +0.108] relative to the pre-exposure code (paired), and both gains are robust to domain clustering. Because those rules were written while the test documents were visible, this gain is an upper bound on what they contribute to unseen data. One of them, the label-anchored tier, also improved presidio-research, a corpus it was never tuned on (Table 10), which is evidence that it generalises. Development-set selection chose the same configuration for both code versions (spaCy on, entropy on, confidence threshold 0.3).

Pre-exposure code against the baselines

The pre-exposure code never saw the benchmark, so it gives the uncontaminated comparison. We re-ran it, Presidio, gitleaks and TruffleHog on the test half, saved per-document predictions, and scored all of them on one shared resample matrix (Table 8). With spaCy, its mandatory recall exceeded Presidio lg by +0.262 [+0.091, +0.436] and Presidio sm by +0.265 [+0.093, +0.441]. Both advantages remain significant when domains are resampled (two-stage +0.255 [+0.026, +0.453] and +0.259 [+0.037, +0.454]). Its character F1 exceeded Presidio lg by +0.129 [+0.033, +0.223] and Presidio sm by +0.138 [+0.047, +0.230]. Only the latter is robust: against Presidio lg the two-stage interval is [−0.024, +0.242]. The pattern tiers alone had higher character F1 than Presidio lg (+0.176 [+0.052, +0.289]) and sm (+0.184 [+0.061, +0.297]) at the document level, but neither difference survives domain clustering. Their mandatory-recall advantage over Presidio lg, +0.176 [−0.001, +0.353], was not significant even at the document level. Against the secret scanners every pre-exposure configuration was ahead under all three schemes; for example, the pattern tiers exceeded gitleaks in mandatory recall by +0.342 [+0.263, +0.419] (two-stage +0.335 [+0.191, +0.479]). No-leak rates did not differ significantly from Presidio sm or lg in either configuration (with spaCy against lg: +0.032 [−0.042, +0.102]).

Table 8 Planned paired comparisons under three bootstrap schemes: median paired difference and 95% interval (2,000 resamples, seed 0, one shared resample matrix per corpus). † marks an interval that includes zero. Pre-exposure SafeStream on RedactionBench is compared with every baseline because it never saw the benchmark.

Comparison	Metric	Document	Cluster	Two-stage
secrets: clusters are 96 rules and 5 negative templates				
Pattern − gitleaks	FR	+.128 [+.073, +.183]	+.123 [+.033, +.206]	+.125 [+.025, +.209]
Pattern − gitleaks	Char F1	+.233 [+.043, +.441]	+.215 [+.013, +.527]	+.226 [+.011, +.538]
Pattern − TruffleHog	FR	+.167 [+.079, +.250]	+.178† [−.010, +.350]	+.184† [−.008, +.363]
Pattern − TruffleHog	Char F1	+.536 [+.310, +.685]	+.529 [+.192, +.740]	+.530 [+.192, +.742]
presidio-research: clusters are 200 templates				
+spaCy − Presidio sm	Char F1	+.051 [+.027, +.076]	+.052 [+.013, +.092]	+.052 [+.013, +.093]
+spaCy − Presidio sm	FR	+.031 [+.004, +.060]	+.032† [−.015, +.077]	+.033† [−.014, +.080]
+spaCy − Presidio lg	Char F1	+.036 [+.011, +.064]	+.036† [−.000, +.075]	+.037† [−.003, +.080]
RedactionBench, pre-exposure SafeStream: clusters are 11 domains				
+spaCy − Presidio lg	Mand. recall	+.262 [+.091, +.436]	+.252 [+.085, +.348]	+.255 [+.026, +.453]
+spaCy − Presidio sm	Mand. recall	+.265 [+.093, +.441]	+.256 [+.104, +.345]	+.259 [+.037, +.454]
+spaCy − Presidio lg	Char F1	+.129 [+.033, +.223]	+.124 [+.006, +.221]	+.123† [−.024, +.242]
+spaCy − Presidio sm	Char F1	+.138 [+.047, +.230]	+.132 [+.031, +.219]	+.132 [+.002, +.242]
+spaCy − Presidio lg	No-leak	+.032† [−.042, +.102]	+.036† [−.096, +.157]	+.031† [−.110, +.169]
Pattern − Presidio lg	Char F1	+.176 [+.052, +.289]	+.167† [−.051, +.298]	+.169† [−.068, +.319]
Pattern − Presidio lg	Mand. recall	+.176† [−.001, +.353]	+.168† [−.252, +.266]	+.164† [−.239, +.373]
Pattern − gitleaks	Mand. recall	+.342 [+.263, +.419]	+.340 [+.217, +.468]	+.335 [+.191, +.479]
Pattern − TruffleHog	Mand. recall	+.571 [+.456, +.666]	+.560 [+.304, +.642]	+.563 [+.315, +.677]
+spaCy − gitleaks	Mand. recall	+.427 [+.350, +.510]	+.433 [+.320, +.561]	+.439 [+.297, +.589]
+spaCy − TruffleHog	Mand. recall	+.656 [+.548, +.749]	+.651 [+.591, +.713]	+.662 [+.526, +.751]
RedactionBench, contamination: exposed minus pre-exposure pattern tiers				
Exposed − pre-exposure	Mand. recall	+.091 [+.044, +.163]	+.094 [+.064, +.159]	+.089 [+.043, +.198]

No-leak documents

The no-leak document rate was low for every system: at most 0.179 [0.105, 0.263] for SafeStream (+spaCy, exposed code), 0.137 for the pre-exposure code with spaCy, 0.084 for the pattern tiers alone, and at most 0.105 for Presidio. In other words, for at least 82% of RedactionBench test documents, every configuration we evaluated left at least one must-redact character in the clear. This metric is deliberately strict. Mandatory recall measures the share of must-redact characters hidden; the no-leak rate requires all of them in a document to be hidden, so a single missed character fails the document. RedactionBench documents are dense: the test half averages about 256 gold spans per document (Table 2), of which 84 on average (median 25) are mandatory.

As an illustration only, if a document had n independent must-redact units each detected with probability p, it would be leak-free with probability p^n, which for n in the hundreds is small even for p close to one. Real misses are correlated (whole entity types missing in a domain), so this is not a model of the data, but it shows why per-character recall in the 0.68–0.76 range is compatible with almost no fully protected documents.

Where the leaks are. We re-ran five systems of Table 7 (exposed pattern tiers, exposed +spaCy, gitleaks, TruffleHog and Presidio lg) on the test half at the commit that produced those rows, reproducing its mandatory recall and no-leak rates exactly, and kept the per-document predictions. The 95 test documents with mandatory gold contain 8,002 mandatory spans. Three observations follow; they are descriptive, from one corpus that had been inspected during development, and carry no confidence intervals.

- *Leaks are whole misses, not boundary errors.* With spaCy, 6,441 mandatory spans were fully redacted, 64 partially and 1,497 not at all; of the 17,122 leaked mandatory characters, 826 (4.8%) lie in partially redacted spans and the rest in spans with no detection. For the pattern tiers alone the share is 3.7% (854 of 22,778). Presidio lg leaks more through partial coverage: 7,354 of its 41,589 leaked characters (17.7%).
- *Few documents are near misses; leaks are concentrated.* With spaCy, 17 documents leaked nothing, 5 more leaked exactly one mandatory span and 7 more leaked two, so fixing one or two spans per document would raise the no-leak rate to at most 29 of 95 (0.305). The median document leaked 6 mandatory spans (interquartile range 2–16), a median of 23% of its mandatory spans, and the 9 documents with the most leaked characters held 67% of them. For the pattern tiers alone the median document leaked 11 spans (52%), and only 2 documents were within two spans of no leak.
- *What is missed.* The corpus carries no entity types, so we grouped spans by a coarse shape heuristic. Capitalised words (largely names) leaked at a rate of 0.90 without a named-entity model and 0.27 with spaCy, which accounts for most of the gap between the two configurations. Spans containing `@` leaked at 0.95 (36 of 38) with spaCy; inspection of a few of them, after all scores were final, showed passwords inside connection strings or quoted configuration values that the e-mail pattern split or that no tier matched. No code was changed in response, and these spans should not be read as a test-set error analysis used for tuning.

Even an oracle that took the union of all five systems' redactions would leave 69 of 95 documents leaking (no-leak rate 0.274), so the low document-level rate is a property of the corpus and of current detectors in general, not only of SafeStream. The low rate is not caused by streaming: by Theorem 1 the streamed output equals the whole-document output, so these leaks are detection misses that a whole-document run would also make.

7.6 RQ6: detection on real system logs

Table 9 gives the results on real `sshd` lines. Every system with an IP pattern, SafeStream and all three Presidio configurations, redacted all 1,349 IP addresses. No

Table 9 Real OpenSSH logs (2,000 lines; Loghub), all gold mandatory. IP and user-name columns give the gold values fully redacted by any detection, whatever its type. FA: share of the 148 lines without gold that received a detection. Point values; the paired differences in the text carry 95% intervals, including two-stage intervals by template (27 clusters).

System	Char P	Char R	Char F1	IP addresses	User names	FA
SafeStream pattern	.730	.786	.757	1,349 / 1,349	0 / 1,142	.000
SafeStream +spaCy (sm)	.548	.790	.647	1,349 / 1,349	21 / 1,142	.155
Presidio regex	.661	.787	.719	1,349 / 1,349	4 / 1,142	.034
Presidio sm	.331	.790	.466	1,349 / 1,349	18 / 1,142	1.000
Presidio lg	.421	.789	.549	1,349 / 1,349	14 / 1,142	.993
gitleaks	.000	.000	.000	0 / 1,349	0 / 1,142	.000
TruffleHog	.000	.000	.000	0 / 1,349	0 / 1,142	.000

system redacted user names: SafeStream's pattern tiers redacted none of 1,142, and the best configuration, SafeStream with spaCy, redacted 21. User names in phrases such as `Invalid user webmaster from` carry no shape or label that a pattern tier keys on, and the statistical models rarely tag them. This is the clearest gap that real data exposed. Because user names are nearly half the gold, the fully redacted rate is 0.54–0.55 for every IP-capable system, and only 38–40% of lines with gold were fully protected.

The systems differed in what else they redacted. Presidio's statistical configurations tagged the syslog timestamp of almost every line as a date, so they flagged 99–100% of the lines without gold, against none for SafeStream's pattern tiers and 0.155 with spaCy. As a result, SafeStream's character F1 exceeded Presidio sm by +0.291 [+0.286, +0.295] and Presidio lg by +0.208 [+0.202, +0.213], and both differences hold under template clustering (two-stage +0.284 [+0.205, +0.351] and +0.207 [+0.130, +0.275]). With spaCy, the advantage over Presidio sm and lg with their own models also holds (two-stage +0.182 [+0.089, +0.273] and +0.101 [+0.007, +0.215]). Against Presidio's pattern recognisers the character-F1 difference, +0.038 [+0.031, +0.045], does not survive template clustering ([−0.001, +0.122]). One precision caveat applies to every IP-capable system alike. In 385 lines the remote host is a reverse-DNS name that embeds the client's IP address (`5.36.59.76.dynamic-...`). The frozen labels exclude hostnames, but all of these systems redacted the embedded address, so their precision is understated, arguably correctly, since the hostname reveals the address. The labels were not changed after scoring. These are one system's logs and two entity types, so they complement rather than replace a broader real-world evaluation (Section 8.4).

7.7 Ablation

Table 10 isolates the components. The entropy tier carried most of the credential recall: without it, character F1 on secrets fell from 0.842 to 0.383. The label-anchored tier mattered on presidio-research, where several identifiers carry labels. The contextual scorer's own detections changed no result on these corpora; its confidence adjustment was kept in every variant. Alone, the deterministic tier reached a character F1 of 0.380 on secrets and 0.237 on presidio-research, and the entropy tier alone 0.822 on secrets. The union rule raised recall by small and mostly significant amounts at no significant

Table 10 Ablation on the test splits (pure-Python backend, identical output to the Rust core). Paired differences against the full pattern pipeline, with 95% document-level intervals.

Change	secrets	presidio-research	RedactionBench
Character F1, full pattern pipeline minus variant			
Remove entropy tier	+.455 [.242, .641]	.000	+.156 [.104, .224]
Remove label-anchored tier	.000	+.030 [.019, .044]	+.004 [.002, .006]
Remove contextual detections	.000	.000	.000
Union rule versus greedy resolution (with spaCy), union minus greedy			
Character recall	+.006 [.000, .017]	+.005 [.002, .008]	+.009 [.004, .016]
FR or mandatory recall[1]	+.013 [.000, .032]	+.001 [.000, .002]	+.012 [.004, .022]

[1]Fully redacted rate (FR) for secrets and presidio-research; mandatory recall for RedactionBench.

cost in F1 on secrets (+0.000 [−0.003, +0.005]) and RedactionBench (+0.003 [−0.002, +0.008]); on presidio-research its F1 difference was +0.004 [+0.001, +0.007]. The ablation was not planned and its intervals are document-level only.

8 Discussion

8.1 What the results establish, and what they do not

Established under stated assumptions: chunked and parallel processing introduce no detection difference relative to the whole-document pipeline, for detectors that respect their declared bounds (Section 5). *Measured:* leaks caused by naive and overlapping chunking on every corpus, including real system logs, and none with SafeStream in any chunking tried; flat working memory across a 256-fold range of input sizes on one input shape; near-linear scaling to four cores; on held-out splits, a higher fully redacted rate for credentials than gitleaks on a corpus of rule examples, robust to rule clustering and to disabling gitleaks' filters, and higher character F1 than TruffleHog with or without its filters; and higher character F1 than Presidio with the same spaCy model on a synthetic corpus, robust to template clustering. *Measured on real logs:* on 2,000 real SSH lines, complete IP-address redaction, no user-name redaction by any system, and no false alarms from the pattern tiers, where Presidio's statistical configurations flagged nearly every line. *Not established:* effectiveness on real-world artefacts beyond these two entity types in one system's logs; document-level protection adequate for public release; robustness to adversarial obfuscation.

8.2 When is the approach useful despite a low no-leak rate?

Bounded memory and equivalence do not make redaction complete; they remove one specific failure mode, the leak introduced by chunking itself, and make large inputs processable at all. The value of that depends on the detection coverage for the values that matter in a given deployment. The evidence supports three uses and argues against a fourth.

1. *Defence in depth for structured secrets in machine-generated text.* Where the sensitive values are credentials and structured identifiers with recognisable shapes (logs, configuration dumps, continuous-integration output), the pattern tiers are where

coverage is highest in our data, and the guarantee ensures that splitting a multi-gigabyte log does not make coverage worse. Even here more than half of the test secrets were not fully redacted, so the tool reduces exposure rather than eliminating it.

2. *Pre-filtering before human review or controlled sharing.* When output is reviewed, or shared only with parties already bound by agreements, a high-precision filter that reliably processes large inputs reduces the volume of exposed values without being the last line of defence.
3. *Inputs that cannot otherwise be processed.* Whole-document redaction needed about 28 times the input size in memory in our measurements, which rules it out for multi-gigabyte files on ordinary hosts.
4. *Not supported: sole control for public release of free text.* With no-leak rates below 0.18 on RedactionBench and low free-text recall without a named-entity model, the results do not justify releasing redacted free-text documents publicly on the strength of this system alone.

A deployment should set its own acceptance criteria before relying on the system, for example: per-category recall measured on a representative, authorised sample of the target artefacts; zero tolerated leaks for specified credential types on that sample; a review step for document classes with free text; and periodic re-measurement as formats change. None of these was measured here.

8.3 Threats to validity

External validity. presidio-research is synthetic and template-generated; the secrets positives are rule-author examples and the negatives are generated from five templates; RedactionBench is the only corpus with real-world seeding, and it is contaminated. The corpora are English-only. The only real-log accuracy result covers 2,000 lines of one SSH server and two entity types, with gold derived from template labels checked by one author rather than double annotation; it says nothing about credentials in real logs. *Construct validity.* The secrets gold spans are Nosey Parker's capture groups, so credentials adjacent to a labelled secret (for example a client identifier next to its secret) count as false positives; RedactionBench's contextual labels have low human agreement according to its authors [33]. *Internal validity.* The secrets test half was scored before and after one fix (Section 7.4); RedactionBench was inspected during development. *Statistical validity.* The planned comparisons were tested under document, cluster and two-stage resampling, and Table 8 shows which differences depend on the independence assumption. Eleven RedactionBench domains are few clusters for a bootstrap, so those intervals are approximate. The ablation intervals are document-level only, and no multiplicity correction was applied. *Comparison fairness.* The scanners' filters target example-like strings, which is what the secrets corpus contains; with them disabled, gitleaks closed most of the character-F1 gap (Table 6), so that margin should not be claimed. *Environment.* All runs used one virtual machine.

8.4 Evaluation plan for real-world evidence

Ranked by expected value per unit of effort:

1. *Real credential-bearing text with clear provenance:* public continuous-integration logs or issue attachments from repositories whose licences permit research use, or logs from an organisation that authorises the study. Safeguards: never test discovered credentials against live services; store raw text encrypted with access limited to annotators; report discovered live secrets to their owners through responsible disclosure; publish only annotations, hashes and aggregate results, not secret values; obtain ethics review where required; double annotation with agreement reported.
2. *Transformer-based detectors*, evaluated on corpora they were not trained on, with an empirical streamed-versus-whole-document check, because the theorems do not cover a model whose output depends on its whole input window.
3. *A second machine* for throughput and memory, to separate implementation effects from the virtual machine.

9 Conclusion

Chunked redaction need not trade correctness for scale. With detector-declared bounds on span length and context, an overlap that covers every detector's span plus its right context makes the streamed result equal to the whole-document result, a seam cross-check makes the parallel result equal to the sequential one, and the working buffer does not grow with the input. On the test corpora, naive chunking leaked values a whole-document pass redacts and overlapping windows did not remove every leak, while SafeStream's output was identical to the whole-document output in every chunking tried. In our measurements memory stayed flat from 8 MiB to 2 GiB and throughput scaled 3.9-fold on four cores. These properties remove chunking as a source of leaks; they do not make detection complete. On held-out data, two margins survived cluster-robust resampling and the scanner sensitivity run: the pattern tiers fully redacted more credentials than gitleaks, and with a spaCy model SafeStream had higher character F1 than Presidio with the same model on a synthetic corpus. On 2,000 real SSH log lines every IP-capable system redacted all IP addresses and none redacted user names, while SafeStream's pattern tiers raised no false alarms where Presidio's statistical configurations flagged nearly every line. Other margins did not survive, fewer than one RedactionBench document in five was fully protected by any configuration, and two of the three corpora are synthetic. The system is therefore suited to reducing exposure in large machine-generated artefacts as one layer among several, not to certifying documents for public release. The next step is evaluation on authorised real-world text (Section 8.4).

Acknowledgements. The authors gratefully acknowledge Sri Mata Amritanandamayi Devi (Amma), Chancellor of Amrita Vishwa Vidyapeetham, for her inspiration and for providing financial support towards the Article Processing Charges (APC) of this publication. The authors also thank the maintainers of Microsoft Presidio, Nosey Parker and RedactionBench for making their software and benchmark datasets publicly available.

Declarations

- **Funding.** The authors gratefully acknowledge the financial support provided by Sri Mata Amritanandamayi Devi (Amma), Chancellor of Amrita Vishwa Vidyapeetham, towards the Article Processing Charges (APC) of this publication.
- **Competing interests.** The authors declare no competing financial or non-financial interests.
- **Ethics approval and consent to participate.** Not applicable: no human participants were involved and the authors collected no data. presidio-research is synthetic, the secrets corpus consists of example strings from an open-source rule set together with generated negatives, and RedactionBench is a public dataset released by its authors under CC BY 4.0, which they describe as seeded from real-world sources. All processing was local; no excerpt of any document is published here, and no credential was sent to a live service (TruffleHog ran without verification).
- **Consent for publication.** Not applicable.
- **Data availability.** presidio-research, Nosey Parker and Loghub are public (Loghub is free for research use with citation of [34]); RedactionBench is available from its authors under CC BY 4.0. Generated non-secrets are produced by the released code with a fixed seed. Split manifests, raw results and per-document predictions (character offsets only, no corpus text) are in the public repository under `research/results/`.
- **Code availability.** Source code and the evaluation harness, including the scripts `benchmarks/cluster_bootstrap.py` and `benchmarks/scanner_sensitivity.py`, are available at https://github.com/MounishSenisetty/SafeStream-Redactor.
- **Author contributions.** Mounish Senisetty: conceptualisation, methodology, software architecture, formal analysis, validation, investigation, data curation, visualisation, writing (original draft) and writing (review and editing). Ashrith Reddy N: software implementation, validation, investigation and writing (review and editing). Sandhiya Kennedy: software implementation, validation, investigation and writing (review and editing). Parvathy R: supervision and writing (review and editing). All authors read and approved the final manuscript.
- **Use of generative AI.** During the preparation of this work the authors used the large language model Claude (Anthropic) to help implement evaluation scripts, draft early manuscript text, convert LaTeX markup to the journal template, edit language and check bibliographic references. After using this tool, the authors reviewed and revised all code, experimental results and text, and take full responsibility for the content.

Appendix A Proofs of Theorems 2 and 3 and implementation mapping

Throughout, positions are absolute offsets in T, the conditions of Theorem 1 hold unless stated otherwise, and $L_{\max} = \max_d L_d \le w$.

A.1 A streaming engine started mid-text

The parallel engine runs Algorithm 1 on a substring of T with a read-only left context. Lemma 1 covers that case.

Lemma 1 *Let $X = T[x_0{:}x_1]$ and let the engine start with $P = T[x_0{:}x_0 + h]$ as already-emitted context and stream $T[x_0 + h : x_1]$, where $h = w$, or $h < w$ and $x_0 = 0$. Then it emits exactly the spans of $\Pi(X)$ that start at or after $x_0 + h$.*

Proof The proof of Theorem 1 uses only two properties of P: it holds the $\min(w, p - x_0)$ characters preceding B, and the window ends at the end of the text being processed. Both hold here with X in place of T. Spans of the first window that start inside P are dropped, as in every later pass. □

Lemma 2 (Transfer to the full text) *With $X = T[x_0{:}x_1]$, let s be a span of detector d that belongs to $\Pi(X)$ or to $\Pi(T)$, with $i - C_{\text{left},d} \geq x_0$ or $x_0 = 0$, and $j + C_{\text{right},d} \leq x_1$ or $x_1 = |T|$. Then $s \in \Pi(X)$ if and only if $s \in \Pi(T)$.*

Proof Apply A2 with $a = x_0$ and $b = x_1$ in T; in X these positions are its two edges. This gives $\delta = -x_0$ in the notation of A2, and an edge of X that coincides with an edge of T satisfies the edge clause. □

A.2 Proof of Theorem 2

Let the split points be $0 = a_0 < a_1 < \cdots < a_k = |T|$, with $a_{r+1} - a_r \geq 4w$; `plan_ranges` enforces this, and coarsens the split or falls back to one range otherwise. Worker r processes $X_r = T[x_0^r : x_1^r]$ with $x_0^r = \max(0, a_r - 2w)$ and $x_1^r = \min(|T|, a_{r+1} + w)$. Its first w characters, $T[a_r - 2w : a_r - w]$, are read-only context, and streaming starts at $a_r - w$; worker 0 has no context and starts at 0. By Lemma 1, worker r obtains D_r, the spans of $\Pi(X_r)$ that start at or after $a_r - w$.

A *straddler* of a seam a is a span with $i < a < j$. Worker r reports the straddler of a_r (its start seam) and the straddler of a_{r+1} (its end seam) found in D_r, or reports that none exists. It emits $T[\ell_r : u_r]$, where ℓ_r is the end of its start-seam straddler or a_r, and u_r is the end of its end-seam straddler or a_{r+1}. Together with that text it emits the spans of D_r contained in $[\ell_r, u_r)$.

Step 1: in the window that matters, worker r sees $\Pi(T)$. Let s be a span of detector d with $a_r - w \leq i < a_{r+1}$ that belongs to D_r or to $\Pi(T)$. On the left, $i - C_{\text{left},d} \geq a_r - 2w \geq x_0^r$, because $C_{\text{left},d} \leq w$. On the right, $j \leq i + L_d < a_{r+1} + L_d \leq a_{r+1} + w - C_{\text{right},d}$, so $j + C_{\text{right},d} \leq x_1^r$, or $x_1^r = |T|$. By Lemma 2, $s \in D_r \Leftrightarrow s \in \Pi(T)$.

Step 2: both neighbours report the same straddler. A straddler of a_r starts after $a_r - L_d \geq a_r - w$, and a straddler of a_{r+1} starts before a_{r+1}, so both lie in the range of Step 1. Spans that start before $a_r - w$ are dropped as context. Such a span ends at most $L_{\max} \leq w$ characters after its start, that is, at or before a_r, so it cannot cross a_r. Each straddler found by worker r is therefore the unique straddler of $\Pi(T)$ at that

seam, if one exists (uniqueness follows from disjointness). Worker $r+1$ applies Step 1 to its own start seam a_{r+1} and obtains the same span. The reports agree, so the valve passes, and $u_r = \ell_{r+1}$.

Step 3: each part is the sequential redaction of its slice. Every span emitted by worker r lies in $[\ell_r, u_r)$. It starts at or after $\ell_r \geq a_r$, and before a_{r+1}: if there is an end-seam straddler, any span starting at or after a_{r+1} and ending by u_r would overlap it; otherwise $u_r = a_{r+1}$. By Step 1 the emitted spans are exactly the spans of $\Pi(T)$ inside $[\ell_r, u_r)$. No cut point ℓ_r lies strictly inside a span of $\Pi(T)$, because it is either a seam with no straddler or the end of a span of $\Pi(T)$, and $\Pi(T)$ is disjoint. The slices $[\ell_r, u_r)$ therefore partition T without cutting any span. Redaction acts on each span separately (Definition 1), so the concatenated parts equal the redaction of T under $\Pi(T)$. By Theorem 1 that is the sequential output.

Step 4: timing of the cuts. A worker writes text only once the cuts that can affect it are fixed. It decides ℓ_r once its stream has passed $a_r + w$. By then every straddler of a_r, which ends before $a_r + L_{\max}$, has been emitted by the streaming engine. It flushes only text below the current position minus $2w$. The stream never extends beyond $a_{r+1} + w$, so this text lies below $a_{r+1} - w$, and the end cut $u_r \geq a_{r+1}$ cannot affect it. This proves (i).

Proof of (ii). This part uses no assumption on the detectors. For each seam, the coordinator compares the left worker's end report with the right worker's start report and redacts the whole file sequentially on any mismatch. If all reports match, each u_r equals ℓ_{r+1}, because both are computed from the same reported straddler end. The written parts then cover $[0, |T|)$ as consecutive, non-overlapping slices, and every character of T is emitted exactly once, redacted or not. A non-splittable text encoding, a single worker or a small file also take the sequential path. When the conditions of Theorem 1 fail, agreeing parts can still contain different detections from the sequential output; this is why an overlap below $w^\star$ is refused by default. □

A.3 Proof of Theorem 3

After an emission. The retained buffer is $B[e{:}]$. If no span straddles the planned boundary, $|B| - e = w$. Otherwise $e = i^* > |B| - w - L_d$, so $|B| - e < w + L_{\max}$. In both cases the retained buffer holds fewer than $w + L_{\max} \leq w + m$ characters.

Before a detection pass. Chunks are appended until $|B| \geq w + m$. Before the last of them, $|B| < w + m$, and that chunk adds at most c characters, so a pass starts with $|B| < w + m + c$. The proof of Theorem 1 shows that the pinned branch, which would let B grow further, is never taken when $m \geq w \geq L_{\max}$. The final buffer obeys the same bound. The window $P \cdot B$ adds $|P| \leq w$, which gives $2w + m + c$. The detections of a window are pairwise disjoint and lie inside it, so their number is at most its length.

Parallel workers. Each worker runs one streaming engine and holds back emitted segments until they lie $2w$ behind its stream position, together with the detections of that suffix. This adds at most $2w$ characters plus one segment of at most $w + m + c$ characters. The bound does not depend on $|T|$, and k workers need at most k times as much. □

A.4 Implementation mapping

Table A1 Proof steps and the code that discharges them (module paths under `src/safestream_redactor/`).

Proof step	Code and tests
Declared $(L_d, C_{\text{left},d}, C_{\text{right},d})$ and $\max(L_d + C_{\text{right},d}, C_{\text{left},d})$	`detectors/base.py`: `StreamingRequirements`, `required_overlap`
$w^\star$ (Eq. 1); refusal below it	`engine.required_overlap`; `Redactor.min_safe_overlap`
Window $P \cdot B$, spans inside P dropped	`engine.stream_windows`, `window_detect`
Planned boundary and retreat	`emit_upto = len(buffer) - overlap`; `_retreat_boundary`
Span longer than w fails closed	`_check_overlong` (`strict_spans`)
Coalescing to m; pinned branch	`min_emit`, `need` in `stream_windows`
Disjoint detection set	`entities.resolve_overlaps`
Span-local rewrite	`policy.RedactionPolicy.apply`, `render`
Ranges of at least $4w$	`parallel.plan_ranges`
Context and padding of X_r	`parallel._redact_range`, `_read_chars_before`, `_read_chars_after`
Straddler reports; cuts ℓ_r, u_r	`parallel._find_straddler`; `start_cut`, `end_cut`
Holdback of $2w$	`flush(cursor - 2 * ov)` in `_redact_range`
Seam valve and fallback	`redact_file_parallel`: `left.end_seam != right.start_seam`
Empirical corroboration	`tests/test_streaming.py`, `test_properties.py`, `test_parallel.py`, `test_streaming_contract.py`, `test_formal_constants.py`, `test_memory.py`

References

[1] Alneyadi, S., Sithirasenan, E., Muthukkumarasamy, V.: A survey on data leakage prevention systems. Journal of Network and Computer Applications **62**, 137–152 (2016) https://doi.org/10.1016/j.jnca.2016.01.008

[2] Meli, M., McNiece, M.R., Reaves, B.: How bad can it git? Characterizing secret leakage in public GitHub repositories. In: Network and Distributed System Security Symposium (NDSS) (2019). https://doi.org/10.14722/ndss.2019.23418

[3] Krause, A., Klemmer, J.H., Huaman, N., Wermke, D., Acar, Y., Fahl, S.: Pushed by accident: A mixed-methods study on strategies of handling secret information in source code repositories. In: 32nd USENIX Security Symposium, pp. 2527–2544 (2023). https://www.usenix.org/conference/usenixsecurity23/presentation/krause

[4] Microsoft: Presidio: Data Protection and De-identification SDK. https://github.com/microsoft/presidio. Version 2.2.355 evaluated. Accessed 11 October 2026 (2026)

[5] Gitleaks contributors: Gitleaks. https://github.com/gitleaks/gitleaks. Version 8.28.0 evaluated. Accessed 11 October 2026 (2026)

[6] Truffle Security: TruffleHog. https://github.com/trufflesecurity/trufflehog. Version 3.90.8 evaluated. Accessed 11 October 2026 (2026)

[7] Basak, S.K., Cox, J., Reaves, B., Williams, L.: A comparative study of software secrets reporting by secret detection tools. In: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (2023). https://doi.org/10.1109/ESEM56168.2023.10304853

[8] Ptacek, T.H., Newsham, T.N.: Insertion, evasion, and denial of service: Eluding network intrusion detection. Technical report, Secure Networks, Inc. (January 1998)

[9] Wang, X., Hong, Y., Chang, H., Park, K., Langdale, G., Hu, J., Zhu, H.: Hyperscan: A fast multi-pattern regex matcher for modern CPUs. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 631–648 (2019). https://www.usenix.org/conference/nsdi19/presentation/wang-xiang

[10] Intel Corporation: Introduction to Hyperscan. https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-hyperscan.html. Accessed 11 October 2026 (2019)

[11] Neamatullah, I., Douglass, M.M., Lehman, L.-w.H., Reisner, A., Villarroel, M., Long, W.J., Szolovits, P., Moody, G.B., Mark, R.G., Clifford, G.D.: Automated de-identification of free-text medical records. BMC Medical Informatics and Decision Making **8**, 32 (2008) https://doi.org/10.1186/1472-6947-8-32

[12] Norgeot, B., Muenzen, K., Peterson, T.A., Fan, X., Glicksberg, B.S., Schenk, G., Rutenberg, E., Oskotsky, B., Sirota, M., Yazdany, J., Schmajuk, G., Ludwig, D., Goldstein, T., Butte, A.J.: Protected health information filter (Philter): accurately and securely de-identifying free-text clinical notes. npj Digital Medicine **3**, 57 (2020) https://doi.org/10.1038/s41746-020-0258-y

[13] Stubbs, A., Uzuner, Ö.: Annotating longitudinal clinical narratives for de-identification: The 2014 i2b2/UTHealth corpus. Journal of Biomedical Informatics **58**(Suppl), 20–29 (2015) https://doi.org/10.1016/j.jbi.2015.07.020

[14] Dernoncourt, F., Lee, J.Y., Uzuner, Ö., Szolovits, P.: De-identification of patient notes with recurrent neural networks. Journal of the American Medical Informatics Association **24**(3), 596–606 (2017) https://doi.org/10.1093/jamia/ocw156

[15] Microsoft: presidio-research: Toolbox for developing and evaluating PII detectors. https://github.com/microsoft/presidio-research. Synthetic dataset `synth_dataset_v2.json`, revision 0cb3650. Accessed 11 October 2026 (2026)

[16] Lison, P., Pilán, I., Sánchez, D., Batet, M., Øvrelid, L.: Anonymisation models for text data: State of the art, challenges and future directions. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing

(Volume 1: Long Papers), pp. 4188–4203 (2021). https://doi.org/10.18653/v1/2021.acl-long.323

[17] Pilán, I., Lison, P., Øvrelid, L., Papadopoulou, A., Sánchez, D., Batet, M.: The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization. Computational Linguistics **48**(4), 1053–1101 (2022) https://doi.org/10.1162/coli_a_00458

[18] Tjong Kim Sang, E.F., De Meulder, F.: Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In: Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, pp. 142–147 (2003). https://aclanthology.org/W03-0419

[19] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186 (2019). https://doi.org/10.18653/v1/N19-1423

[20] Sanh, V., Debut, L., Chaumond, J., Wolf, T.: DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108 (2019). https://doi.org/10.48550/arXiv.1910.01108

[21] Zaratiana, U., Tomeh, N., Holat, P., Charnois, T.: GLiNER: Generalist model for named entity recognition using bidirectional transformer. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 5364–5376 (2024). https://doi.org/10.18653/v1/2024.naacl-long.300

[22] Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-Hashing for Message Authentication. RFC 2104 (1997). https://doi.org/10.17487/RFC2104

[23] Sweeney, L.: k-anonymity: A model for protecting privacy. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems **10**(5), 557–570 (2002) https://doi.org/10.1142/S0218488502001648

[24] Sinha, V.S., Saha, D., Dhoolia, P., Padhye, R., Mani, S.: Detecting and mitigating secret-key leaks in source code repositories. In: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories (MSR), pp. 396–400 (2015). https://doi.org/10.1109/MSR.2015.48

[25] Saha, A., Denning, T., Srikumar, V., Kasera, S.K.: Secrets in source code: Reducing false positives using machine learning. In: 2020 International Conference on COMmunication Systems & NETworkS (COMSNETS) (2020). https://doi.org/10.1109/COMSNETS48256.2020.9027350

[26] Feng, R., Yan, Z., Peng, S., Zhang, Y.: Automated detection of password leakage

from public GitHub repositories. In: Proceedings of the 44th International Conference on Software Engineering (ICSE) (2022). https://doi.org/10.1145/3510003.3510150

[27] Basak, S.K., Neil, L., Reaves, B., Williams, L.: SecretBench: A dataset of software secrets. In: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR) (2023). https://doi.org/10.1109/MSR59073.2023.00053

[28] Shannon, C.E.: A mathematical theory of communication. Bell System Technical Journal **27**(3), 379–423 (1948) https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

[29] Praetorian: Nosey Parker. https://github.com/praetorian-inc/noseyparker. Rule set at revision 2e6e7f3. Accessed 11 October 2026 (2026)

[30] Muthukrishnan, S.: Data streams: Algorithms and applications. Foundations and Trends in Theoretical Computer Science **1**(2), 117–236 (2005) https://doi.org/10.1561/0400000002

[31] Thompson, K.: Regular expression search algorithm. Communications of the ACM **11**(6), 419–422 (1968) https://doi.org/10.1145/363347.363387

[32] Luhn, H.P.: Computer for Verifying Numbers. US Patent 2,950,048 (1960)

[33] Brynjólfsson, S., Jayakrishnan, S., Sali, E., Purwar, D., Aggarwal, M.: RedactionBench. arXiv:2606.18782 (2026). https://doi.org/10.48550/arXiv.2606.18782

[34] Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A large collection of system log datasets for AI-driven log analytics. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), pp. 355–366 (2023). https://doi.org/10.1109/ISSRE59848.2023.00071

[35] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Chapman & Hall/CRC, New York (1993). https://doi.org/10.1201/9780429246593

[36] Field, C.A., Welsh, A.H.: Bootstrapping clustered data. Journal of the Royal Statistical Society: Series B (Statistical Methodology) **69**(3), 369–390 (2007) https://doi.org/10.1111/j.1467-9868.2007.00593.x